MODEL TRAINING DOCUMENTATION

Fetal Health Classification — Model Training Guide

Random Forest • Scikit-learn • Flask Integration • Joblib

Document Type	Model Training Documentation
Project	Fetal Health Classification Using CTG Data
Training Notebook	fetal-health-classification-model.ipynb
Saved Model File	optimized_fetal_health_model.pkl
Algorithm	Random Forest Classifier (scikit-learn)
Input Features	21 Numeric CTG Features
Output	Normal • Suspect • Pathological
Document Status	Final • For Mentor Review

1. Development Environment

Before running the training notebook, make sure the following are installed. All of these are standard Python packages that can be installed with pip.

Package	What it is used for	Minimum Version
`Python`	The main programming language for the whole project	3.7+
`Flask`	Running the web application that serves predictions	2.0+
`scikit-learn`	Building, training, and evaluating the classifier	1.0+
`pandas`	Loading the CSV dataset and managing the data table	1.3+
`numpy`	Numeric operations and array handling	1.21+
`joblib`	Saving and loading the trained model file	1.0+
`Jupyter Notebook`	Running the interactive training notebook	6.0+

2. Training Notebook

All the training work is done inside a single Jupyter notebook:

```
fetal-health-classification-model.ipynb
```

This notebook goes through everything in one place: loading the data, exploring it, training the model, checking how it performs, and saving the final output. It is meant to be run top to bottom in order. Each section has comments explaining what is happening and why.

Keeping everything in one notebook makes it easier to see the full picture and reproduce the results without having to piece together separate scripts.

3. Training Workflow

The training process follows these steps in order:

1. Load the dataset from the CSV file at dataset/fetal_health.csv into a pandas DataFrame.
2. Check the data structure, column names, and look for any missing values or obvious problems.
3. Separate the features (the 21 CTG columns) from the target label (the fetal_health column).
4. Split the data into a training set and a test set using stratified sampling to keep the class proportions balanced.
5. Create a Random Forest classifier and fit it on the training set.
6. Run predictions on the test set and calculate accuracy, precision, recall, and F1-score.
7. Save the trained model to optimized_fetal_health_model.pkl using joblib so it can be used by the web app.

Each step is a separate cell in the notebook so it is easy to re-run individual parts if something needs to be adjusted.

4. Why Random Forest

Random Forest was chosen as the main model for a few reasons:

- It handles numeric features well without needing a lot of preprocessing, which suits this dataset since all 21 features are already numeric.
- It is an ensemble method that trains many decision trees and combines their outputs, which helps reduce overfitting compared to a single tree.
- It gives probability estimates for each class, not just a hard prediction, which makes it possible to show a confidence level in the app.

- It generally works well on datasets of this size (around 2,000 rows) without needing a lot of tuning to get decent results.

A basic Decision Tree was also tested as a baseline. The Random Forest performed better on the test set, especially for the minority classes, so it was kept as the final model.

5. Feature Handling

The dataset comes with 21 numeric CTG features that are used directly as model inputs. No manual feature engineering was done for this version of the project. Here is how the features are handled:

- All 21 columns are kept as-is. There is no need for categorical encoding since everything is already a number.
- The feature order matters - the order of columns in the training data must exactly match the order of inputs in the web form. Any mismatch would give wrong predictions.
- No feature scaling was applied because Random Forest does not require it. The algorithm works with raw values.

The 21 input features cover things like the baseline fetal heart rate, number of accelerations and decelerations per second, short-term and long-term variability, and histogram statistics of the FHR distribution. The exact feature names can be seen in the dataset and the notebook.

6. Evaluation Strategy

The model is evaluated on a test set that was held out during training. A few different metrics are used because accuracy alone does not tell the full story when the classes are imbalanced.

Metric	Why it matters for this project
Accuracy	Gives a general idea of overall performance but can be misleading if the model mostly just predicts Normal.
Precision	Shows how many of the predicted positives for each class were actually correct.
Recall	Especially important for the Pathological class - we want the model to catch those cases even if it means a few false alarms.
F1-Score	Balances precision and recall. Useful for comparing classes where one is much rarer than the others.
Confusion Matrix	Shows exactly where the model is making mistakes, which classes it confuses with each other.

Because the dataset has a lot more Normal examples than Suspect or Pathological, the results are reported for each class separately and also as a weighted average. This gives a more honest picture of how well the model actually works.

7. Saving the Trained Model

Once training is done, the model is saved to a file using joblib:

```
joblib.dump(model, 'optimized_fetal_health_model.pkl')
```

This .pkl file is what the Flask app loads when it starts up. Saving it separately from the training code means the app does not have to retrain every time it runs, which keeps load times fast.

The file should be placed in the root of the project folder so that app.py can find it. If the file is missing or moved, the app will throw an error on startup.

8. How the Model Connects to the Web App

The Flask application (app.py) loads the saved model once when the server starts. When a user submits the prediction form, here is what happens:

8. The 21 input values from the form are collected and converted into a list.
9. That list is turned into a single-row pandas DataFrame, with columns in the same order as the training data.
10. The model's predict() method returns the class label (Normal, Suspect, or Pathological).
11. The model's predict_proba() method returns the probability for each class, which is shown as the confidence score.
12. The result page displays both the predicted class and the confidence breakdown.

The key thing to get right is the column order. If the feature columns in the DataFrame do not match what the model was trained on, the predictions will be wrong. The feature names used in training are saved alongside the model to make this easier to check.

9. How to Retrain the Model

To retrain from scratch, follow these steps:

13. Create a virtual environment and activate it.
14. Install all dependencies by running: pip install -r requirements.txt
15. Make sure the dataset file is at dataset/fetal_health.csv.
16. Open the notebook: fetal-health-classification-model.ipynb
17. Run all cells from top to bottom using Kernel > Restart & Run All.
18. Check that optimized_fetal_health_model.pkl has been created or updated in the project folder.

The whole process should take under a minute on a standard laptop. If anything goes wrong, the notebook prints the evaluation results at each step so it is easy to see where something failed.

10. Notes for Future Improvements

This version of the training pipeline is kept simple on purpose - the focus was on getting a working model into the app. There are a few things that could make it better in a future version:

- Adding cross-validation (e.g. k-fold) instead of a single train/test split would give a more reliable estimate of how the model generalises.
- Trying techniques to handle class imbalance, like class weighting or SMOTE, could improve recall on the Suspect and Pathological classes.
- A feature importance chart would be useful for showing which CTG measurements the model relies on most, and would make the app more informative.
- Hyperparameter tuning with grid search or random search could squeeze out a bit more performance from the Random Forest.
- Adding precision and recall per class to the app's metrics page would give users a more complete view of what the model can and cannot do.

End of Model Training Documentation